

Class & Struct – gói code thành đối tượng

Vấn đề

Tại sao bài này quan trọng? Khi bài toán phức tạp dần (Hasher cho xâu, Segment Tree ở Vàng, đồ thị, ...), code rời rạc bằng các mảng toàn cục sẽ trở nên rối, khó tái sử dụng, dễ sai. Gói tất cả với dữ liệu và các thao tác liên quan ở chung một chỗ vào một class – code gọn, rõ ràng, và dễ đem sang bài khác.

Sau bài này bạn sẽ biết: cách viết một class / struct đơn giản, cách dùng nó nhiều lần trong code, và cách áp dụng vào những cấu trúc dữ liệu thực tế như Hasher (băm xâu, bài 7).

1. Vấn đề: code "phẳng" với biến toàn cục

Giả sử bạn cần viết một stack tự cài đặt. Cách "trực tiếp":

C++

```
int st[100005], top_idx = 0;

void push(int x) { st[top_idx++] = x; }
int pop() { return st[--top_idx]; }
int top() { return st[top_idx - 1]; }
bool empty() { return top_idx == 0; }
```

Hỏi: nếu bài cần 2 stack thì sao? Phải copy mọi thứ ra `st1`, `top_idx1`, `push1`, ... – code dài, lằng, dễ sai.

Đây là lúc class/struct giải cứu.

2. Struct trong C++ – gộp dữ liệu

`struct` là kiểu dữ liệu gộp nhiều biến thành một "khối". Mặc định, mọi thành viên là public (truy cập tự do).

C++

```
struct Point {
    int x, y;
};

Point p;
p.x = 3;
p.y = 5;

Point q = {1, 2}; // Khởi tạo gọn

cout << p.x << ' ' << q.y;
```

Khi nào dùng `struct`? Bất cứ khi nào bài toán có đối tượng có nhiều thuộc tính: điểm (x, y) , cạnh (u, v, w) , học sinh (tên, điểm), ...

struct có thể có hàm

C++

```
struct Point {
    int x, y;

    int distSq(Point other) { // Hàm thành viên
        int dx = x - other.x, dy = y - other.y;
        return dx * dx + dy * dy;
    }
};

Point a = {1, 2}, b = {4, 6};
cout << a.distSq(b); // 25
```

Bên trong hàm thành viên, `x` và `y` tự động trở tới thành viên của đối tượng đang gọi.

3. class trong C++ – struct "có tường ngăn"

`class` giống `struct` nhưng mặc định mọi thành viên là `private` (không truy cập từ ngoài). Bạn dùng `public`: để công khai phần "API".

Trong CP, cả `struct` và `class` đều phổ biến – sự khác nhau chỉ là quyền truy cập mặc định. Nhiều CP-er dùng `struct` để khỏi phải gõ `public`:

C++

```
class Stack {
private:
    vector<int> data;

public:
    void push(int x) { data.push_back(x); }
    void pop() { data.pop_back(); }
    int top() { return data.back(); }
    bool empty() { return data.empty(); }
    int size() { return data.size(); }
};

Stack a, b; // Hai stack doc lap!
a.push(5); a.push(7);
b.push(3);
cout << a.top() << ' ' << b.top(); // 7 3
```

Giờ bạn có thể tạo bao nhiêu stack tùy ý – mỗi cái có dữ liệu riêng biệt.

Constructor – khởi tạo

Constructor là hàm đặc biệt được gọi khi tạo đối tượng. Tên trùng với tên **struct** / **class**, không có kiểu trả về:

C++

```
struct Counter {
    int value;

    Counter() { value = 0; } // Constructor mac dinh
    Counter(int start) { value = start; } // Constructor co tham so

    void inc() { value++; }
};

Counter a; // value = 0
Counter b(100); // value = 100
b.inc(); // 101
```

4. Class trong Python

Python dùng từ khóa **class**. Method đầu tiên đặc biệt là **__init__** (constructor). Tham số đầu tiên của mọi method luôn là **self** (tương đương **this** ngầm trong C++).

Python

```
class Stack:
    def __init__(self):
        self.data = [] # Thuoc tinh cua doi tuong

    def push(self, x):
        self.data.append(x)

    def pop(self):
        return self.data.pop()

    def top(self):
        return self.data[-1]

    def empty(self):
        return len(self.data) == 0

a = Stack()
b = Stack()
a.push(5); a.push(7)
b.push(3)
print(a.top(), b.top()) # 7 3
```

Lưu ý quan trọng: trong Python, mọi truy cập thuộc tính trong method phải qua **self**. Viết **data** mà không có **self.** sẽ tạo biến cục bộ mới (bug âm thầm).

5. Ví dụ thứ hai: gói các thuộc tính của một đối tượng

Một dạng dùng `struct` rất phổ biến trong CP: gom các thuộc tính của một thực thể. Giả sử bài có n học sinh, mỗi học sinh có tên và điểm; cần sắp theo điểm giảm dần, đồng điểm thì theo tên.

C++

```
struct Student {
    string name;
    int score;

    bool operator<(const Student& o) const {
        if (score != o.score) return score > o.score; // Diem giam
        return name < o.name; // Ten alphabet
    }
};

vector<Student> v(n);
for (auto& s : v) cin >> s.name >> s.score;
sort(v.begin(), v.end());
for (auto& s : v) cout << s.name << ' ' << s.score << '\n';
```

Python

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

v = [Student(*input().split()) for _ in range(n)]
for s in v: s.score = int(s.score)
v.sort(key=lambda s: (-s.score, s.name)) # Diem giam, ten alphabet
for s in v: print(s.name, s.score)
```

(Cách viết Python "thuần CP" thường chỉ dùng tuple (name, score) cho gọn nhưng class rõ nghĩa hơn khi đối tượng có nhiều trường.)

6. Nhìn trước: Hasher sẽ trông thế nào?

Ở bài 7 ta sẽ học String Hashing – kỹ thuật băm xâu để so sánh đoạn con xâu trong $O(1)$. Khi đó, bạn có thể đóng gói toàn bộ Hasher vào một `struct` / `class` để dùng đi dùng lại:

C++

```
struct Hasher {
    vector<long long> h, pw;

    Hasher(const string& s) { /* se hoc bai 7 */ }
    long long get(int l, int r) { /* hash doan [l..r] se hoc bai 7 */ }
};
```

```
};

// Su dung - ngan va ro:
Hasher H(s);
if (H.get(l1, r1) == H.get(l2, r2)) {
    /* hai doan bang nhau (xac suat rat cao) */
}
```

Python

```
class Hasher:
    def __init__(self, s):
        # se hoc bai 7
        pass

    def get(self, l, r):
        # se hoc bai 7
        pass

H = Hasher(s)
if H.get(l1, r1) == H.get(l2, r2):
    # hai doan bang nhau (xac suat rat cao)
    pass
```

Đoạn code trên chỉ là khung mẫu – không chạy được vì các phương thức để trống. Đừng cố compile/chạy ngay; bạn sẽ điền đầy đủ ở bài 7. Điểm cần nhớ ở đây là phong cách: thay vì 4-5 mảng toàn cục `h[]`, `pw[]`, `build()`, `get()`, ta gói thành một `class`. Lợi ích:

- Có thể tạo nhiều Hasher độc lập trong cùng một bài (ví dụ: hash của hai xâu khác nhau).
- Code `H.get(l, r)` tự nói lên ý nghĩa.
- Copy `class` sang bài khác là một khối duy nhất.

7. Khi nào nên (và không nên) dùng class trong CP?

Nên dùng class khi:

- Bài cần nhiều instance của cùng một cấu trúc (nhiều Hasher, nhiều stack, ...).
- Cấu trúc dữ liệu phức tạp (Hasher, Segment Tree, Trie, ...) – code "phẳng" sẽ rối.
- Bạn muốn tái sử dụng code giữa các bài.

Không nên dùng class khi:

- Bài chỉ cần một mảng đơn giản với 1-2 thao tác – viết hàm rời cũng được.
- Constructor làm chậm (Python) – nếu thực sự cần tốc độ đỉnh, có thể quay lại code "phẳng".

Trong CP nói chung: học thuộc một class Hasher và một class Segment Tree rồi dán vào bài khi cần – đây là phong cách của hầu hết CP-er có kinh nghiệm.

8. Bẫy thường gặp

- Quên `self`. trong method Python – tạo biến cục bộ, không thay đổi thuộc tính.
- `struct` tham trị: trong C++, truyền `struct` vào hàm mặc định là copy – chậm cho cấu trúc lớn. Dùng tham chiếu `Hasher&` để truyền.
- Constructor nặng: code đặt mọi thứ trong constructor có thể gây TLE nếu khởi tạo nhiều instance. Cân nhắc khởi tạo lười (lazy).
- Độ quy đệ quy giới hạn stack: Python có giới hạn đệ quy mặc định (~ 1000) – với cây/đồ thị lớn, có thể tăng bằng `sys.setrecursionlimit(...)` hoặc cài lại bằng vòng lặp. Sẽ thấy chi tiết ở các bài có DFS sâu (BFS/DFS lưới bài 9, cây bài 10).

9. Tổng kết

KHÁI NIỆM	C++	PYTHON
Gộp dữ liệu	<code>struct Foo { };</code>	<code>class Foo:</code>
Hàm thành viên	Trong <code>struct</code> / <code>class</code>	Method với <code>self</code>
Khởi tạo	Constructor <code>Foo()</code>	<code>__init__(self)</code>
Truy cập	<code>obj.field</code>	<code>obj.field</code>

Class/struct là bước nhảy về phong cách lập trình – khi đã quen, code của bạn sẽ trông gọn gàng và "chuyên nghiệp" hơn rất nhiều. Hãy luyện tập ngay từ bài 7 (Hasher) viết Hasher dưới dạng class!